

VUE.JS · LECTURE 1

Vue.js Crash Course

Introduction to Vue.js Fundamentals — a hands-on lecture for intermediate JavaScript developers ready to build reactive, component-driven user interfaces.

v3.5.39



What We'll Cover Today

01

Vue.js Fundamentals

What it is, history, and framework philosophy

03

Components & Setup

Component anatomy, environment options, Options API

05

Modifiers & Computed Properties

Input modifiers and cached reactive logic

02

MPA vs. SPA & Virtual DOM

Architecture models and Vue's rendering engine

04

Data Binding & Directives

Mustache, v-bind, v-model, and core directives

06

Live Demo & Takeaways

Hands-on walkthrough and key concept recap

BEFORE WE BEGIN

Prerequisites

You should be comfortable with these JavaScript fundamentals before diving in:



JavaScript Core

Loops, functions, objects, arrays, arrow functions, destructuring, and higher-order methods



Events & DOM

Event listeners, DOM manipulation, and the Fetch API for HTTP requests



NPM

Node Package Manager — installing dependencies and running scripts

What Is Vue.js?



Framework Philosophy

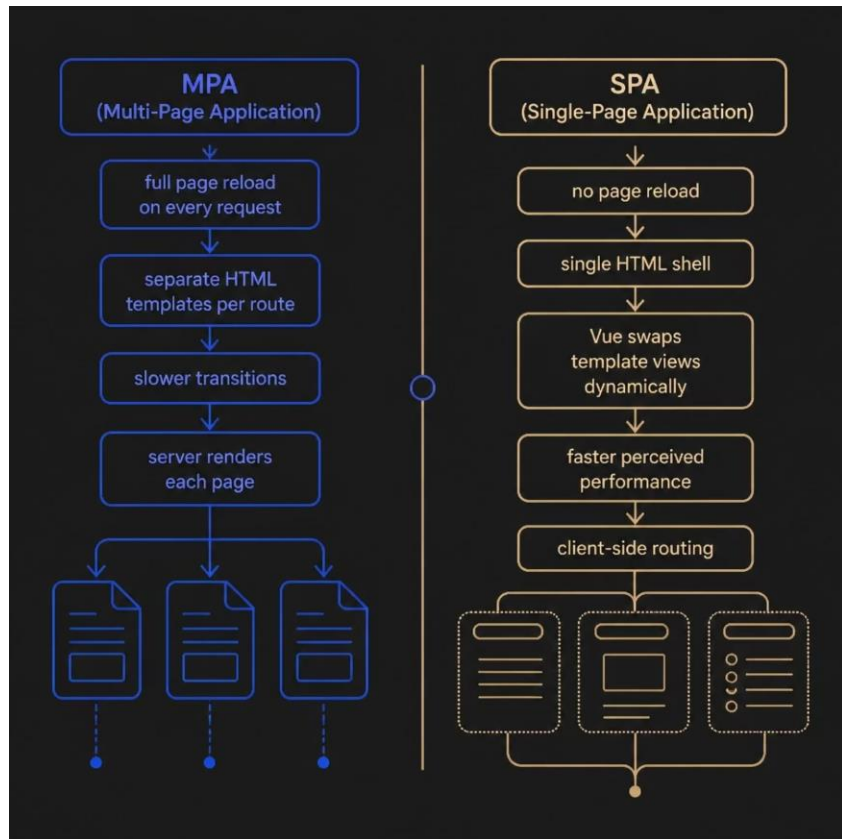
A **JavaScript framework** provides structure for building UIs , handling state, routing, and rendering so you focus on features, not plumbing.

Vue.js is a **progressive framework** for building user interfaces and SPAs. Created in **2013** by Evan You, it's designed to be simple, flexible, and incrementally adoptable.

The Role of Framework

- Enhance UI /UX
- **Organizations** : Component based
- **Performance** : Optimized for performance with feature such as the virtual DOM
- **Reactive data-binding** : UI updates automatically when data changes
- **Component-based architecture** : build UIs from reusable pieces
- **Incrementally adoptable** : drop into any project, scale as needed
- **Modularity** : Allow Developer to break down the apps into smaller , reusable components.

MPA vs. SPA



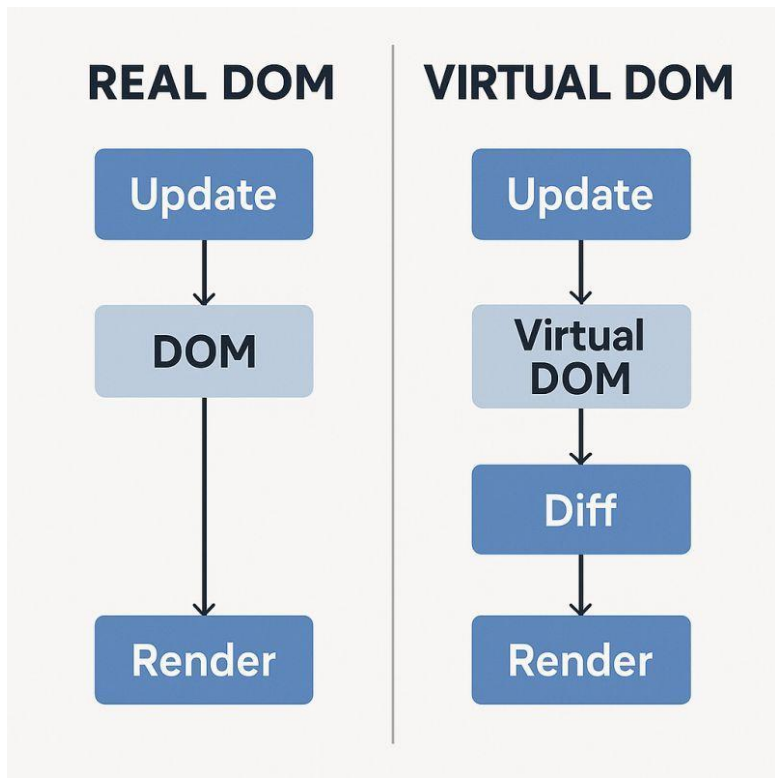
MPA vs SPA (Why Vue Fits SPA)

Vue.js is built for **Single-Page Applications**. The page loads once, then Vue updates only the content instead of reloading the whole page. This makes navigation faster and smoother.

MPA: Better for content-heavy or SEO-focused websites.

SPA: Better for interactive web applications.

The Virtual DOM



Vue creates a lightweight virtual copy of the UI in memory.

When data changes, Vue compares the old and new virtual trees, then updates only the parts that actually changed in the Real DOM.

In **Vue 3**, updates are even faster.

Besides the Virtual DOM, Vue also uses **Compiler Optimizations** and its **Reactivity System** to detect changes more efficiently and reduce unnecessary Virtual DOM work.

Why Vue?

- Simplicity & Approachability.
- Easy to learn.
- Flexibility
- Performace & Size
- Component-Based Architecture
- Active Community & Rich EcoSystem

[Libraries , tools , plugins , Meta Framework : Nuxt.js(for build server side render app) & Gridsome(Static side generation)]

Vue Components

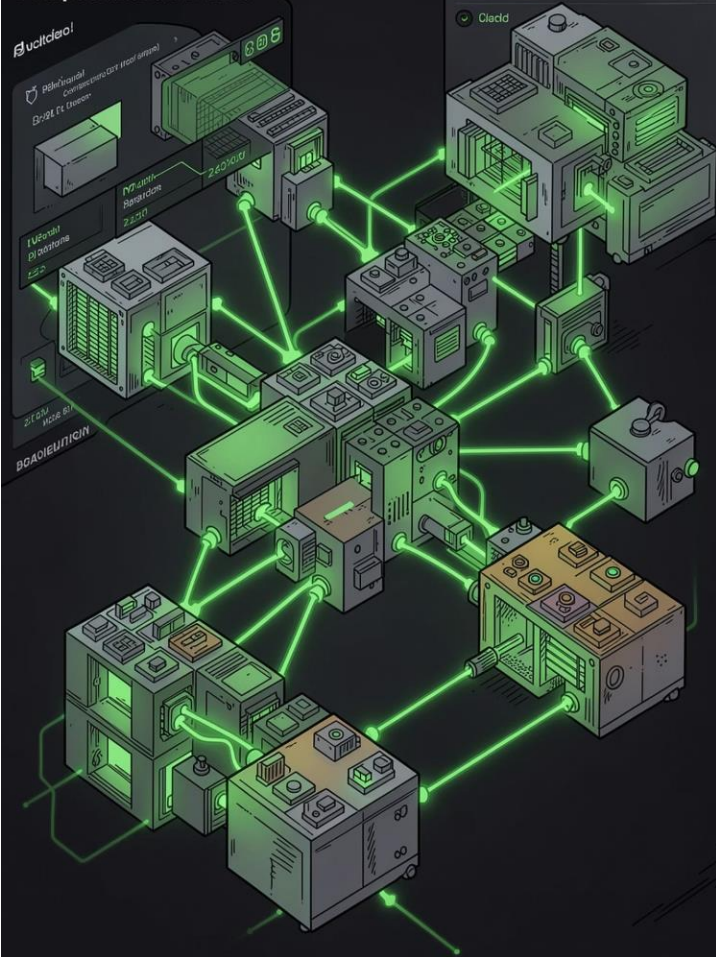
Components are **reusable, self-contained units** each bundles its own logic, template, and scoped styles. Vue supports both the **Options API** (classic) and **Composition API** (modern).

```
<template> //template
  <h1>{{ message }}</h1>
  <button @click="greet">Say Hi</button>
</template>

<script> //logic
export default {
  data() {
    return { message: 'Hello from Vue.js!' }
  },
  methods: {
    greet() { alert(this.message) }
  }
}
</script>

<style scoped> //scoped style
  h1 { color: #42b883; }
</style>
```

Component Builblocks



Setup Options & Data Binding

[vuejs](https://vuejs.org)

Getting Started

- **CDN** — drop-in script tag, zero build tools
- **create-vue** — official Vite-based scaffolder (recommended)
- **Vue CLI** — legacy tooling, still in use
- **Nuxt.js / Gridsome** — SSR and static-site frameworks

Three Binding Modes

- **{{ }}** — Text interpolation (mustache)
- **v-bind** — One-way attribute binding
- **v-model** — Two-way form input binding

Directives & Modifiers

Core Directives

- **v-text / v-html** — render text or raw HTML
- **v-bind (:)** — bind attributes dynamically
- **v-on (@)** — attach event listeners
- **v-if / v-else** — conditional rendering
- **v-show** — toggle visibility via CSS
- **v-for** — render lists from arrays

Common Modifiers

- **.lazy** — sync on change, not input
- **.trim** — strip whitespace
- **.prevent** — call event.preventDefault()
- **.enter / .ctrl** — key-specific listeners
- **.once** — trigger handler only once

Computed Properties & Live Demo

Computed vs. Methods

Computed properties are **cached** — they re-evaluate only when dependencies change, unlike methods which run on every render.

```
computed: {  
  reversed() {  
    return this.msg.split('').reverse().join('')  
  }  
}
```

Live Demo Covers

- Rendering messages with `{{ }}`
- Conditional rendering: **v-if** / **v-show**
- List rendering: **v-for** with a people array
- Two-way binding: **v-model** on inputs
- Counter with **@click** events
- Computed property with **getter** / **setter**



Key Takeaways: Reactivity, components, directives, binding, and computed properties are Vue's core. Next up: lifecycle hooks, props, and state management.